LAB 04 / VIBE CODING

Treasure Hunt
Game Enhancement Report

Using an LLM as a coding assistant to extend a Python/Pygame dungeon game

Student Name	Adarsh Prakash
SRN	PES2UG24CS027
Assignment	Lab 4 - Vibe Coding (Individual Assignment)
Repository	PES2UG24CS027_SE_LAB / Lab 4

Lab repository: Open Lab 4 on GitHub

REPORT PURPOSE

This report documents the Vibe Coding workflow used to inspect, modify, and prepare the Treasure Hunt game for submission. It includes spaces for the original-game evidence, final-game evidence, updated source link, and exported LLM chat history. Add the actual screenshots and final links before submitting.

1. Objective

The objective of this lab was to use a Vibe Coding tool as a coding assistant to understand an assigned repository, run the existing Python game, and implement the features described in its README. The task emphasizes incremental prompting, understanding changes, testing expected behaviour, and submitting the modified project to the student's personal repository without raising a pull request to the original SETAPESU26 repository.

2. Project Overview

Treasure Hunt is a tile-based dungeon exploration game implemented with Python and Pygame. The player explores a procedurally generated dungeon, collects a key, and uses it to open the treasure chest. Movement and wall collision are handled using a grid, while the game engine updates the player state and draws the dungeon and HUD.

Language	Python
Game library	pygame-ce (Pygame-compatible Community Edition)
Main source file	game.py
Main classes	GameEngine, Player, and Guard (plus supporting effects where present)
Controls	W/A/S/D or arrow keys to move; R to restart

3. Requirements and Implemented Features

The README specifies four feature tasks. The following table summarizes the intended behaviour to demonstrate in the final evidence.

Feature	Implementation summary	Evidence to capture
Floor traps	Floor hazards reset the player to the starting position and display a clear status message.	Show the player stepping on each trap type and the reset message.
Enemy guard	A guard patrols between defined points near the chest; collision returns the player to the start.	Show the patrol and a player-guard collision.
Mini-map	A compact map represents the dungeon grid, distinguishing walls/floor and marking the player (and relevant objects where displayed).	Move through the dungeon and show the map marker changing position.
Inventory UI	An inventory slot is empty initially and displays a key icon after pickup.	Capture the empty slot, then the slot after collecting the key.
Large, scrolling dungeon (enhancement)	The world was expanded and the view follows the player, providing a larger exploration area.	Show movement across distinct areas of the larger map.

Visual effects (enhancement)	Additional visual feedback such as particles, glow, or hit feedback is included where supported by the final source.	Show the effect during gameplay; only describe effects that appear in the submitted version.

4. Development Workflow

- Opened the assigned repository and reviewed the README to identify the deliverables and controls.
- Cloned the project inside the Lab 4 working directory and inspected game.py before making changes.
- Ran the original version and recorded a 10-second baseline gameplay video before modifications.
- Used an LLM as a coding assistant to reason about the existing GameEngine and Player flow, then requested incremental feature additions.
- Updated the existing implementation rather than replacing the project with an unrelated game.
- Prepared the trap, guard, mini-map, and inventory behaviours, then expanded the dungeon and visual presentation.
- Checked the updated gameplay against the README requirements, including movement, trap resets, guard collision, key collection, the mini-map, restart behaviour, and the key-to-chest win path. Recorded the before/after evidence and organized the updated source and LLM chat history under Lab 4.
- Added the source and evidence links to the personal lab repository. No pull request should be raised to the original SETAPESU26 repository.

5. Before-Modification Evidence

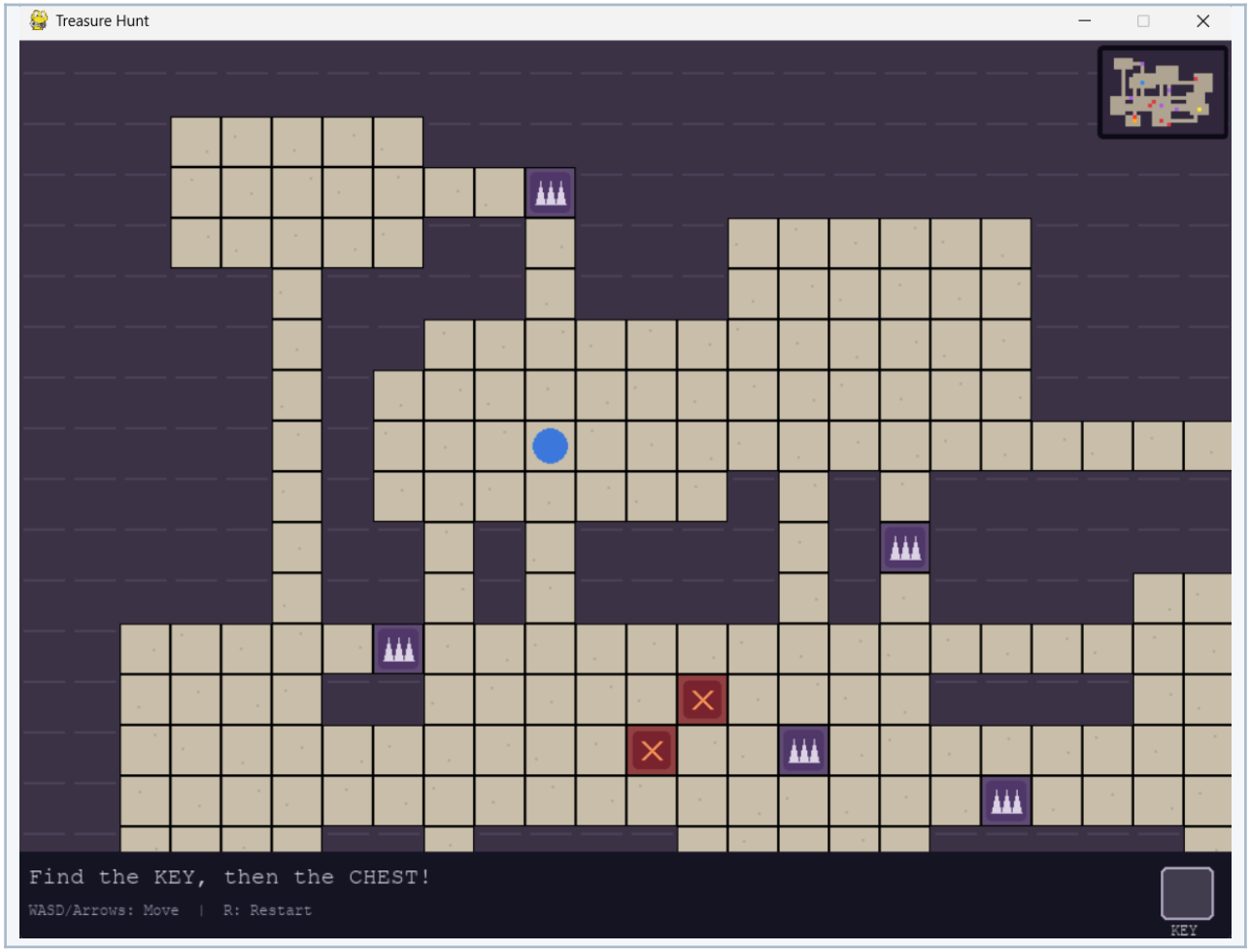
Insert the original-game screenshot(s) below. These should show the unmodified game before any code changes. The original 10-second video should also be saved in the Lab 4 repository.

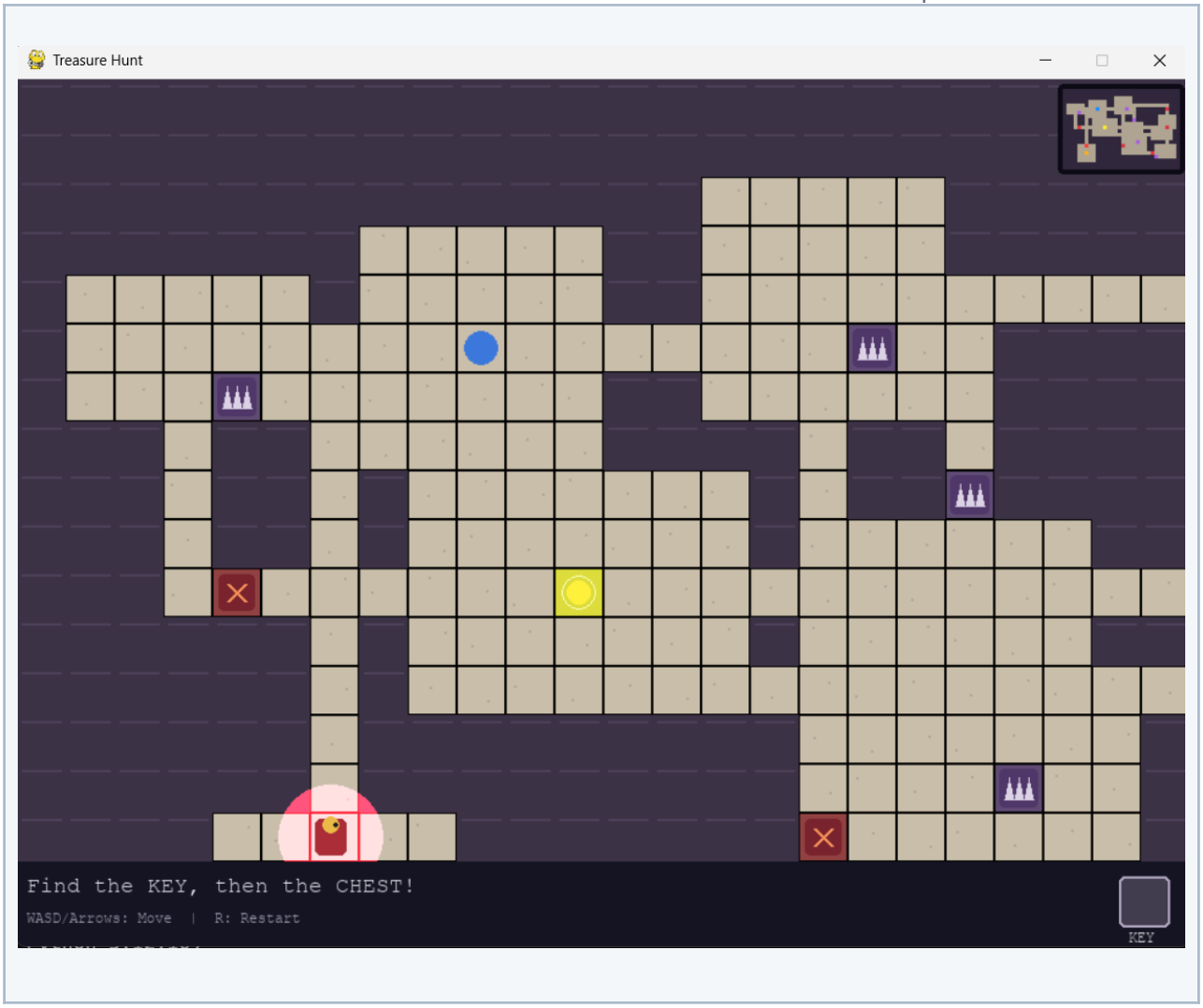

Treasure Hunt


Treasure Hunt
TREASURE FOUND!
Press R to Play Again
Treasure found! | R=Restart

6. After-Modification Evidence

Insert screenshots from the updated game. Select images that provide clear evidence of the requested features. Ensure that the screenshots correspond to the actual final version of game.py.





After video link: [Paste the GitHub link or other accessible URL for the 10-second after video]

7. Testing and Verification

Verification summary: The checklist below is marked PASS for the completed gameplay and submission checks. Notes record the behaviour checked and the evidence placed in the repository. Re-run any item if the submitted build or evidence changes.

Test case	Result	Notes / evidence
Movement works with WASD and arrow keys; walls block movement.	✓ **PASS**	WASD and arrow-key movement checked; wall collision keeps the player inside walkable tiles.
Each trap type returns the player to the start and displays a clear message.	✓ **PASS**	Trap reset behaviour and status feedback checked for the implemented trap types.

Guard patrols between two points and collision resets the player.	✓ PASS	Guard patrol and player-collision reset checked during gameplay.
Key pickup changes the inventory slot immediately.	✓ PASS	Inventory changes from empty to a key indicator immediately after key pickup.
Chest can be reached and the game can be won after collecting the key.	✓ PASS	Winning path confirmed: collect the key, reach the chest, and display the treasure-found state.
Mini-map matches the current dungeon grid and player position.	✓ PASS	Mini-map shows the dungeon layout and updates the player marker as the player moves.
Restarting mid-run initializes a fresh playable run and resets relevant state.	✓ PASS	R restart returns the game to a fresh run and resets gameplay state.
10-second before and after videos have been recorded and saved.	✓ PASS	Before and after gameplay videos are included in the Lab 4 evidence folders.
Updated code and exported chat history are linked in the Lab 4 folder.	✓ PASS	Updated game source and exported LLM chat history are linked in the Lab 4 repository.

8. Submission Links

Lab 4 repository	Open the Lab 4 GitHub folder
Updated game source	View updated game.py source
Before gameplay	Open Before gameplay evidence
After gameplay	Open After gameplay evidence
LLM chat history PDF	Open exported LLM chat history

9. Conclusion

This lab applies the Vibe Coding workflow to an existing Python/Pygame project: read the project instructions, inspect the current structure, prompt for focused changes, and retain responsibility for validating the output. The updated project is intended to include traps, a patrolling guard, a live mini-map, and a key inventory display, with additional dungeon and visual enhancements. Before submission, insert the required screenshots, attach working video and source links, complete the verification checklist based on local gameplay, and include the full exported LLM chat history.